iSMILE 2026 · 20–21 NOVEMBER · SULAIMANI

The registration database — how it works, every detail

What is stored, why, who can see it, how money and tickets are protected, and what is deleted when.

18	241	13	5
tables	columns	ready-made lists	staff roles

Made from `backend/database/schema.sql` on branch **backend**, 6 October 2026.
Database: MySQL / MariaDB, utf8mb4 (Kurdish and Arabic stored exactly as typed). Server: api.ismile.krd (currently in **test** mode).

1. The one rule above all

A person is registered only after the payment company confirms the exact amount — or the Owner gives a free ticket. An unpaid form is never a registration: it never appears in the registrations, the lists or the counts, and it is deleted automatically if nobody pays.

2. The journey of one registration

1 · Form sent
Website or office phone call → row in `checkouts` (status open)

›

2 · Payment started
Row in `payments`; the server works out the amount

›

3 · Psoola confirms
Signed webhook, logged in `webhook_log`; amount must match exactly

›

4 · Registered
Row in `registrations` + `tickets` (T26-00001…)

›

5 · Email
Queued in `emails`, sent within a minute, in English

Website registration
The form is kept for **24 hours** while the person pays. They must tick "tickets are non-refundable" (saved as `terms_accepted_at`).

Phone registration (office)
Staff fill in the form; the caller gets a "Pay now" email. The link works for **7 days** (Settings → pay link days). Staff cannot type a price: the server always uses the website prices.

Wrong amount or paid twice
The payment is marked `mismatch` or `duplicate`: **no ticket**, held for Finance, who mark it `kept` after review. iSmile does not refund.

Free ticket
Only the Owner can give one, and must write a reason (`comp_reason`). The ticket's source is `complimentary` with no payment.

3. Prices and money

Item	Price	Where it is set
Professional ticket	**40,000 IQD**	Website file `data/tickets.json` (Site content › Ticket prices). api.ismile.krd copies it from ismile.krd every minute.
Student ticket	**30,000 IQD**	
Lunch day 1	not set yet (0)	
Lunch day 2	not set yet (0)	

- Prices are **not** stored in the database; the amount for each payment is calculated by the server at the moment of paying and saved in `payments.amount_expected`.
- A price of 0 means "not set": nobody can pay for that item. A lunch with price 0 cannot be chosen.

- Every price change is noticed within 5 minutes, written to the audit log and emailed to the Owner.
- Money is always whole Iraqi dinars (no decimals). Workshop and sponsor money is recorded separately (sections 5).

4. Design rules the database enforces by itself

1. **Links are checked** (foreign keys): no payment for a form that does not exist, no ticket without a registration, no action by an unknown staff member.
2. **Fixed lists** (ENUM): status, ticket type, language, payment method… any other value is refused.
3. **Rules** (CHECK): e.g. a "paid" payment must have the exact confirmed amount; a student must have a university; a free ticket needs a reason; age 16–120.
4. **Unique**: one ticket per person, one ticket per payment, each reference once, each Psoola payment recorded once, one booking per person per workshop.
5. **History is kept**: registrations, payments and tickets are marked (cancelled, removed, disabled), not deleted. Staff accounts are disabled, never deleted.
6. **Text**: utf8mb4 everywhere, so Kurdish and Arabic names are stored exactly as typed.
7. **Times**: Asia/Baghdad local time, set by the server.

5. Every table, every column

Colour legend for rules: UNIQUE no duplicates allowed · LINK must point to a real row · RULE the database refuses anything else. “required” = the field cannot be empty.

5.1 Admin accounts

admin_users

Admin accounts and roles

WHAT IT HOLDS
Staff who can sign in to the admin, with their role.

WRITTEN BY
Owner (Users page)

READ BY
Sign-in, every admin page

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`email`	VARCHAR(190) **required**	stored lower-case
`name`	VARCHAR(120) **required**	Name
`role`	ENUM('owner','registration','finance','content','checkin') **required**	Staff role
`password_hash`	VARCHAR(255) **required**	bcrypt/argon, never the password
`totp_secret`	VARCHAR(64) may be empty	the phone-code key
`totp_enabled`	TINYINT(1) **required** DEFAULT 0	1 = phone codes are switched on for this account
`totp_last_step`	BIGINT UNSIGNED may be empty	last phone code used: a code works only once
`session_version`	INT UNSIGNED **required** DEFAULT 1	raised on password change / phone reset: signs out old sessions
`failed_logins`	TINYINT UNSIGNED **required** DEFAULT 0	Wrong passwords in a row
`locked_until`	DATETIME may be empty	Sign-in blocked until this time
`last_login_at`	DATETIME may be empty	Last successful sign-in
`disabled_at`	DATETIME may be empty	When the account was switched off (never deleted)
`created_at`	DATETIME **required**	When the row was made

UNIQUE No two rows may share `email`.

RULE Phone codes are either on (1) or off (0).

5.2 Waiting for payment (temporary)

`student_id_photos` Student ID photos (the images themselves)

WHAT IT HOLDS
The student ID pictures themselves, stored inside the database.

WRITTEN BY
Website form / office

READ BY
Registration staff (Photos), Owner

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`image`	MEDIUMBLOB **required**	the photo itself (JPEG)
`mime`	ENUM('image/jpeg') **required** DEFAULT 'image/jpeg'	File type (always JPEG)
`width`	SMALLINT UNSIGNED **required**	Width in pixels
`height`	SMALLINT UNSIGNED **required**	Height in pixels
`bytes`	INT UNSIGNED **required**	File size in bytes
`sha256`	CHAR(64) **required**	fingerprint: the same photo sent twice is visible
`original_name`	VARCHAR(190) may be empty	the file name on the student's phone
`uploaded_at`	DATETIME **required**	When it was uploaded
`uploaded_ip`	VARCHAR(45) may be empty	Internet address of the upload

RULE A photo cannot be empty, and its recorded size must equal the real size of the image.

Fast search on: `sha256`

`checkouts` Forms waiting for payment (temporary, not registrations)

WHAT IT HOLDS
A form that has been sent but NOT paid yet. Temporary.

WRITTEN BY
Website form, office phone registration

READ BY
Payment step, list 13, clean-up job

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`ref`	CHAR(12) **required**	ISM26-XXXXXX; the registration keeps the same reference
`source`	ENUM('website','office') **required** DEFAULT 'website'	Where it came from: website or office form (forms); payment or free ticket (tickets)
`status`	ENUM('open','paid','expired') **required** DEFAULT 'open'	Where it stands (see the allowed values)
`first_name`	VARCHAR(60) **required**	the form, exactly as the registration will be
`father_name`	VARCHAR(60) **required**	Father's name
`grandfather_name`	VARCHAR(60) **required**	Grandfather's name
`phone`	VARCHAR(20) **required**	Phone number (normalised)

Column	Type	Meaning
`email`	VARCHAR(190) **required**	Email address
`city`	VARCHAR(80) **required**	City
`gender`	ENUM('female','male','other','prefer-not') **required**	Gender
`age`	TINYINT UNSIGNED may be empty	Age (optional)
`specialty`	ENUM('gp','spec','omfs','lab','acad','student') **required**	Specialty: GP, specialist, OMFS, lab, academic or student
`lang`	ENUM('en','ar','ku') **required** DEFAULT 'en'	Which website language the person used (English, Arabic or Kurdish). Emails are always English.
`ticket_type`	ENUM('professional','student') **required**	Professional or student ticket
`lunch_day1`	TINYINT(1) **required** DEFAULT 0	Lunch on day 1 (1 = yes)
`lunch_day2`	TINYINT(1) **required** DEFAULT 0	Lunch on day 2 (1 = yes)
`pay_method`	ENUM('visa','mastercard','fib','fastpay') **required** DEFAULT 'visa'	Payment method chosen
`university`	VARCHAR(160) may be empty	students
`ambassador_code`	VARCHAR(40) may be empty	Ambassador code typed by the student
`id_photo_id`	INT UNSIGNED may be empty	the student ID photo (student_id_photos)
`terms_accepted_at`	DATETIME may be empty	ticked "tickets are non-refundable" (website form)
`view_nonce`	CHAR(32) **required**	with the secret: their payment page link
`registration_id`	INT UNSIGNED may be empty	set when paid
`created_by`	INT UNSIGNED may be empty	the staff member, for a phone registration
`created_ip`	VARCHAR(45) may be empty	Internet address the form came from
`created_at`	DATETIME **required**	When the row was made
`expires_at`	DATETIME **required**	after this, it cannot be paid any more

UNIQUE No two rows may share `ref`.

LINK `created_by` must point to an existing row in `admin_users`.

LINK `id_photo_id` must point to an existing row in `student_id_photos`. If that row is deleted, this field is emptied.

RULE Lunch choices are only yes (1) or no (0).

RULE Anyone whose specialty is “student” must have a student ticket.

RULE A student ticket must have a university.

Fast search on: `status, expires_at`

5.3 Registrations (paid people only)

`registrations` Registered people (paid or free ticket only)

WHAT IT HOLDS
Every registered person. A row exists only after payment is confirmed (or a free ticket by the Owner).

WRITTEN BY
Payment confirmation, Owner (free tickets)

READ BY
Registrations, Lists, Check-in, exports

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`ref`	CHAR(12) **required**	ISM26-XXXXXX
`first_name`	VARCHAR(60) **required**	the person
`father_name`	VARCHAR(60) **required**	Father's name
`grandfather_name`	VARCHAR(60) **required**	Grandfather's name
`phone`	VARCHAR(20) **required**	normalised: +9647XXXXXXXXX or +<international>
`email`	VARCHAR(190) **required**	Email address
`city`	VARCHAR(80) **required**	City
`gender`	ENUM('female','male','other','prefer-not') **required**	Gender
`age`	TINYINT UNSIGNED may be empty	Age (optional)
`specialty`	ENUM('gp','spec','omfs','lab','acad','student') **required**	Specialty: GP, specialist, OMFS, lab, academic or student
`lang`	ENUM('en','ar','ku') **required** DEFAULT 'en'	Which website language the person used (English, Arabic or Kurdish). Emails are always English.
`ticket_type`	ENUM('professional','student') **required**	what they chose and paid for
`lunch_day1`	TINYINT(1) **required** DEFAULT 0	Lunch on day 1 (1 = yes)
`lunch_day2`	TINYINT(1) **required** DEFAULT 0	Lunch on day 2 (1 = yes)
`pay_method`	ENUM('visa','mastercard','fib','fastpay') may be empty	NULL for a free ticket
`university`	VARCHAR(160) may be empty	students: kept as a record (no approval step)
`ambassador_code`	VARCHAR(40) may be empty	as typed; unknown codes are shown in Settings
`id_photo_id`	INT UNSIGNED may be empty	the student ID photo; NULL after it is deleted (90 days after the summit)
`id_photo_deleted_at`	DATETIME may be empty	When the student ID photo was deleted
`terms_accepted_at`	DATETIME may be empty	when they accepted "tickets are non-refundable" (proof)
`status`	ENUM('paid','complimentary','cancelled') **required**	where it stands
`possible_duplicate`	TINYINT(1) **required** DEFAULT 0	same phone or email as another registration
`comp_reason`	VARCHAR(255) may be empty	why the Owner gave a free ticket
`notes`	TEXT may be empty	office notes
`view_nonce`	CHAR(32) **required**	with the secret: their own ticket page link
`created_by`	INT UNSIGNED may be empty	record keeping the staff member, for phone / free tickets
`created_ip`	VARCHAR(45) may be empty	Internet address the form came from
`created_at`	DATETIME **required**	when the form was sent
`paid_at`	DATETIME **required**	when the payment was confirmed (or the free ticket given)

Column	Type	Meaning
`updated_at`	DATETIME **required**	When the row was last changed

UNIQUE No two rows may share `ref`.

LINK `created_by` must point to an existing row in `admin_users`.

LINK `id_photo_id` must point to an existing row in `student_id_photos`. If that row is deleted, this field is emptied.

RULE Lunch choices are only yes (1) or no (0).

RULE Anyone whose specialty is “student” must have a student ticket.

RULE A student ticket must have a university.

RULE Age, if given, must be between 16 and 120.

RULE A free ticket (complimentary) must have a written reason.

Fast search on: `status, paid_at`, `email`, `phone`, `ambassador_code`

`ambassadors` Ambassador codes

WHAT IT HOLDS
Ambassador codes, to count students per ambassador.

WRITTEN BY
Owner (Settings)

READ BY
Settings, list 04

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`code`	VARCHAR(40) **required**	The ambassador code
`owner_name`	VARCHAR(120) **required**	The ambassador's name
`university`	VARCHAR(160) may be empty	University (students)
`active`	TINYINT(1) **required** DEFAULT 1	1 = in use
`created_at`	DATETIME **required**	When the row was made

UNIQUE No two rows may share `code`. the collation makes it case-insensitive

5.4 Money and tickets

`payments` Payment attempts

WHAT IT HOLDS
Every payment attempt and what Psoola answered.

WRITTEN BY
Payment step, webhook, 5-minute job

READ BY
Finance (Payments), lists 11 and 12

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`checkout_id`	INT UNSIGNED may be empty	the form being paid (NULL once that is cleaned up)

Column	Type	Meaning
`registration_id`	INT UNSIGNED may be empty	the registration this payment created or belongs to
`gateway`	ENUM('fake','psoola') **required**	fake = the pretend gateway on the test site
`provider_payment_id`	VARCHAR(120) may be empty	Psoola's id for this payment
`method`	ENUM('visa','mastercard','fib','fastpay') may be empty	Payment method
`amount_expected`	INT UNSIGNED **required**	set by the server when the attempt is made
`currency`	CHAR(3) **required** DEFAULT 'IQD'	Currency (IQD)
`amount_confirmed`	INT UNSIGNED may be empty	what Psoola itself confirmed
`status`	ENUM('created','waiting','paid','failed','expired','mismatch','duplicate','kept') **required** DEFAULT 'created'	Where it stands (see the allowed values)
`redirect_url`	TEXT may be empty	mismatch = wrong amount, duplicate = paid twice: no ticket, held for Finance; kept = Finance reviewed it. iSmile does not refund money. Psoola's payment page for this attempt
`last_error`	VARCHAR(500) may be empty	Last error message
`raw_create`	MEDIUMTEXT may be empty	Psoola's answers, kept for disputes
`raw_status`	MEDIUMTEXT may be empty	Psoola's latest status answers, kept for disputes
`checks`	SMALLINT UNSIGNED **required** DEFAULT 0	how often we asked Psoola
`created_at`	DATETIME **required**	When the row was made
`updated_at`	DATETIME **required**	When the row was last changed
`confirmed_at`	DATETIME may be empty	When the payment was confirmed

UNIQUE No two rows may share `gateway, provider_payment_id`. each Psoola payment is recorded once

LINK `checkout_id` must point to an existing row in `checkouts`. If that row is deleted, this field is emptied.

LINK `registration_id` must point to an existing row in `registrations`.

RULE Every payment attempt must be for more than 0 IQD.

RULE A payment can only be “paid” if it belongs to a registration and the amount Psoola confirmed is EXACTLY the expected amount.

Fast search on: `checkout_id, status`, `registration_id`, `status, created_at`, `confirmed_at`

`tickets` Tickets

WHAT IT HOLDS
One ticket per registered person (T26-00001…).

WRITTEN BY
Payment confirmation, Owner

READ BY
Emails, Check-in, list 01

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`registration_id`	INT UNSIGNED **required**	Which registration
`ticket_no`	VARCHAR(16) **required**	T26-00001
`version`	SMALLINT UNSIGNED **required** DEFAULT 1	raised when a name change reissues it; old QR stops working
`source`	ENUM('payment','complimentary') **required**	Where it came from: website or office form (forms); payment or free ticket (tickets)
`payment_id`	INT UNSIGNED may be empty	the payment that paid for it
`issued_by`	INT UNSIGNED may be empty	the Owner, for free tickets
`created_at`	DATETIME **required**	When the row was made
`checked_in_at`	DATETIME may be empty	When they arrived at the door
`checked_in_by`	INT UNSIGNED may be empty	Who checked them in
`cancelled_at`	DATETIME may be empty	When the ticket was cancelled

UNIQUE No two rows may share `registration_id`.
UNIQUE No two rows may share `ticket_no`.
UNIQUE No two rows may share `payment_id`. one payment never makes two tickets
LINK `registration_id` must point to an existing row in `registrations`.
LINK `payment_id` must point to an existing row in `payments`.
LINK `issued_by` must point to an existing row in `admin_users`.
LINK `checked_in_by` must point to an existing row in `admin_users`.
RULE A ticket from a payment must name that payment; a free ticket must have no payment.

Fast search on: `checked_in_at`

5.5 Workshops and sponsors

`workshops` Workshops (managed in the admin)

WHAT IT HOLDS
The workshops, prices and seats. The website cards are made from this table.

WRITTEN BY
Owner (Workshops page)

READ BY
Website, Workshops page, list 05

Column	Type	Meaning
`id`	VARCHAR(40) **required**	short name used in links, e.g. "implant"
`icon`	VARCHAR(20) **required** DEFAULT 'tools'	the picture on the website card
`title_en`	VARCHAR(160) may be empty	empty = "Coming soon" on the website
`title_ar`	VARCHAR(160) may be empty	Title in Arabic (website)

Column	Type	Meaning
`title_ku`	VARCHAR(160) may be empty	Title in Kurdish (website)
`company_en`	VARCHAR(160) may be empty	Company in English
`company_ar`	VARCHAR(160) may be empty	Company in Arabic (website)
`company_ku`	VARCHAR(160) may be empty	Company in Kurdish (website)
`speaker_en`	VARCHAR(160) may be empty	Trainer in English
`speaker_ar`	VARCHAR(160) may be empty	Trainer in Arabic (website)
`speaker_ku`	VARCHAR(160) may be empty	Trainer in Kurdish (website)
`price`	INT UNSIGNED **required** DEFAULT 0	IQD per seat; 0 = price not set yet
`total_seats`	SMALLINT UNSIGNED **required**	the limit
`status`	ENUM('active','hidden') **required** DEFAULT 'active'	hidden = not on the website, no new bookings
`sort_order`	SMALLINT UNSIGNED **required** DEFAULT 0	Position in the list
`created_at`	DATETIME **required**	When the row was made
`updated_at`	DATETIME **required**	When the row was last changed

RULE A workshop has between 1 and 2,000 seats.

RULE A workshop's short name uses only small letters, numbers and dashes (2–40 characters).

Fast search on: `status, sort_order`

`workshop_bookings` Workshop bookings (by phone)

WHAT IT HOLDS
Who is booked on which workshop (booked by phone) and whether they paid.

WRITTEN BY
Registration staff

READ BY
Workshops page, lists 05 and 06

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`registration_id`	INT UNSIGNED **required**	Which registration
`workshop_id`	VARCHAR(40) **required**	which workshop (workshops.id)
`price_agreed`	INT UNSIGNED **required** DEFAULT 0	IQD, fixed when booked
`payment_status`	ENUM('unpaid','paid','complimentary') **required** DEFAULT 'unpaid'	Unpaid, paid or free (complimentary)
`amount_paid`	INT UNSIGNED may be empty	IQD received; must equal price_agreed
`paid_how`	ENUM('cash','transfer','psoola') may be empty	How the money was received
`paid_at`	DATETIME may be empty	When the money was received
`paid_recorded_by`	INT UNSIGNED may be empty	who took / recorded the money
`booked_by`	INT UNSIGNED may be empty	Who booked it
`over_capacity`	TINYINT(1) **required** DEFAULT 0	the Owner booked it although it was full

Column	Type	Meaning
`notes`	VARCHAR(500) may be empty	Office notes
`created_at`	DATETIME **required**	When the row was made
`updated_at`	DATETIME **required**	When the row was last changed
`removed_at`	DATETIME may be empty	a removed booking frees the seat
`removed_by`	INT UNSIGNED may be empty	Who removed it
`active`	TINYINT(1) AS (IF(removed_at IS may be empty, 1, may be empty)) STORED	1 while the booking is active, NULL once removed: lets the database refuse the same person twice on one workshop, while keeping removed history.

UNIQUE No two rows may share `registration_id, workshop_id, active`.

LINK `registration_id` must point to an existing row in `registrations`.

LINK `workshop_id` must point to an existing row in `workshops`.

LINK `booked_by` must point to an existing row in `admin_users`.

LINK `paid_recorded_by` must point to an existing row in `admin_users`.

LINK `removed_by` must point to an existing row in `admin_users`.

RULE Workshop “paid” needs the exact agreed amount, how it was paid and when; “unpaid” and “free” must have no money recorded.

Fast search on: `workshop_id, removed_at`, `registration_id`

sponsor_packages

Sponsor packages and booth types, with prices

WHAT IT HOLDS
Sponsorship packages and booth types with price and places.

WRITTEN BY
Owner (Sponsors page)

READ BY
Website cards, Sponsors page, list 10

Column	Type	Meaning
`id`	VARCHAR(40) **required**	short name, e.g. "gold", "booth-standard"
`kind`	ENUM('sponsor','booth') **required**	sponsorship, or an exhibition booth
`name_en`	VARCHAR(80) **required**	Name in English
`name_ar`	VARCHAR(80) may be empty	Name in Arabic (website)
`name_ku`	VARCHAR(80) may be empty	Name in Kurdish (website)
`subtitle_en`	VARCHAR(160) may be empty	e.g. "Headline partners"
`subtitle_ar`	VARCHAR(160) may be empty	Subtitle in Arabic (website)
`subtitle_ku`	VARCHAR(160) may be empty	Subtitle in Kurdish (website)
`price`	INT UNSIGNED **required** DEFAULT 0	IQD, the list price told on the phone; 0 = not set yet
`places`	SMALLINT UNSIGNED **required** DEFAULT 0	how many can be sold; 0 = no limit
`style`	VARCHAR(30) **required** DEFAULT 'tc-silver'	the colour of the website card
`status`	ENUM('active','hidden') **required** DEFAULT 'active'	Where it stands (see the allowed values)
`sort_order`	SMALLINT UNSIGNED **required** DEFAULT 0	Position in the list

Column	Type	Meaning
`created_at`	DATETIME **required**	When the row was made
`updated_at`	DATETIME **required**	When the row was last changed

RULE A package's short name uses only small letters, numbers and dashes (2–40 characters).

Fast search on: `kind, status, sort_order`

`sponsor_requests` Sponsor and booth requests

WHAT IT HOLDS
Sponsor and exhibition requests from the website, and their progress.

WRITTEN BY
Website sponsor form, staff

READ BY
Sponsors page, lists 07 and 08

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`ref`	CHAR(11) **required**	SPN26-XXXXX
`kind`	ENUM('sponsor','booth') **required**	Type
`package_id`	VARCHAR(40) may be empty	the package they want; NULL = not sure yet
`company`	VARCHAR(160) **required**	who they are (exactly what the website form collects)
`contact_name`	VARCHAR(120) **required**	Contact person
`contact_role`	VARCHAR(120) may be empty	Their job title
`phone`	VARCHAR(20) **required**	Phone number (normalised)
`email`	VARCHAR(190) **required**	Email address
`website`	VARCHAR(190) may be empty	Company website
`city`	VARCHAR(80) may be empty	City
`message`	TEXT may be empty	Message from the form
`lang`	ENUM('en','ar','ku') **required** DEFAULT 'en'	Which website language the person used (English, Arabic or Kurdish). Emails are always English.
`status`	ENUM('new','contacted','agreed','paid','confirmed','declined','waiting_list') **required** DEFAULT 'new'	where it stands
`assigned_to`	INT UNSIGNED may be empty	the team member handling it
`price_quoted`	INT UNSIGNED may be empty	IQD, the amount last told on the phone
`amount_agreed`	INT UNSIGNED may be empty	IQD, the amount they agreed to pay
`amount_paid`	INT UNSIGNED may be empty	IQD, received
`paid_how`	ENUM('cash','transfer','psoola','other') may be empty	How the money was received
`paid_at`	DATETIME may be empty	When the money was received

Column	Type	Meaning
`booth_number`	VARCHAR(20) may be empty	where their booth is (exhibition)
`last_call_at`	DATETIME may be empty	When they were last called
`next_call_at`	DATETIME may be empty	when to call them again
`notes`	TEXT may be empty	Office notes
`created_ip`	VARCHAR(45) may be empty	Internet address the form came from
`created_at`	DATETIME **required**	When the row was made
`updated_at`	DATETIME **required**	When the row was last changed

UNIQUE No two rows may share `ref`.

LINK `assigned_to` must point to an existing row in `admin_users`.

LINK `package_id` must point to an existing row in `sponsor_packages`.

RULE A sponsor can only be Paid or Confirmed with the amount, how and when it was paid.

RULE A sponsor can only be Agreed, Paid or Confirmed once an agreed amount is written.

Fast search on: `status, created_at`, `kind, package_id, status`, `next_call_at`

`sponsor_calls`

Phone calls with sponsors and exhibitors

WHAT IT HOLDS
Every phone call with a sponsor or exhibitor.

WRITTEN BY
Staff (Sponsors page)

READ BY
Sponsors page, list 09

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`request_id`	INT UNSIGNED **required**	Which sponsor request
`called_by`	INT UNSIGNED may be empty	Who made the call
`called_at`	DATETIME **required**	When
`outcome`	ENUM('reached','no_answer','call_back','interested','agreed','declined') **required**	What happened on the call
`amount_quoted`	INT UNSIGNED may be empty	IQD told on this call
`note`	VARCHAR(500) may be empty	Short note
`next_call_at`	DATETIME may be empty	When to call again

LINK `request_id` must point to an existing row in `sponsor_requests`.

LINK `called_by` must point to an existing row in `admin_users`.

RULE A call marked “agreed” must record the amount told.

Fast search on: `request_id, called_at`

5.6 Messages

emails

Email queue

WHAT IT HOLDS
The email queue: every email, its status and delivery attempts. English only.

WRITTEN BY
Payments, office, sponsors, alerts

READ BY
Communications page, sending job

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`kind`	VARCHAR(30) **required**	ticket, pay_now, alert, sponsor_received, sponsor_notify
`registration_id`	INT UNSIGNED may be empty	Which registration
`checkout_id`	INT UNSIGNED may be empty	a "pay now" link for a phone registration
`sponsor_request_id`	INT UNSIGNED may be empty	Which sponsor request
`to_email`	VARCHAR(190) **required**	Who receives it
`data`	TEXT may be empty	extra details as JSON (e.g. a rejection reason)
`status`	ENUM('pending','sent','failed') **required** DEFAULT 'pending'	Where it stands (see the allowed values)
`attempts`	TINYINT UNSIGNED **required** DEFAULT 0	How many times sending was tried
`next_attempt_at`	DATETIME **required**	When to try sending next
`last_error`	VARCHAR(500) may be empty	Last error message
`provider_message_id`	VARCHAR(190) may be empty	The email service's id (Resend), to trace a delivery
`created_at`	DATETIME **required**	When the row was made
`sent_at`	DATETIME may be empty	When it was sent

LINK `registration_id` must point to an existing row in `registrations`.
LINK `checkout_id` must point to an existing row in `checkouts`. If that row is deleted, this row is deleted too.
LINK `sponsor_request_id` must point to an existing row in `sponsor_requests`.

Fast search on: `status, next_attempt_at`, `registration_id, kind`, `sponsor_request_id`, `checkout_id`

webhook_log

Messages from the payment company

WHAT IT HOLDS
Every message received from the payment company, and what was done with it.

WRITTEN BY
Webhook address

READ BY
Finance checks, security

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`received_at`	DATETIME **required**	When it arrived
`ip`	VARCHAR(45) may be empty	Internet address

Column	Type	Meaning
`gateway`	ENUM('fake','psoola') **required**	Which payment gateway (fake on the test site, psoola when live)
`signature_ok`	TINYINT(1) **required** DEFAULT 0	1 = the payment company's signature was correct
`provider_payment_id`	VARCHAR(120) may be empty	Psoola's id for the payment
`outcome`	VARCHAR(80) **required**	e.g. "waiting -> paid", "REJECTED: signature"
`headers`	TEXT may be empty	The message headers, kept for checks
`body`	MEDIUMTEXT may be empty	The message itself, kept for checks

Fast search on: `received_at`, `outcome, received_at`, `provider_payment_id`

5.7 System

`settings` Owner settings

WHAT IT HOLDS
Owner settings (open/closed, capacity, email test mode...).

WRITTEN BY
Owner (Settings page)

READ BY
Whole system

Column	Type	Meaning
`k`	VARCHAR(60) **required**	Setting name
`v`	TEXT may be empty	Setting value
`updated_at`	DATETIME **required**	When the row was last changed

`audit_log` Audit log

WHAT IT HOLDS
Who did what and when. Never edited, never deleted.

WRITTEN BY
Every important action

READ BY
Owner

Column	Type	Meaning
`id`	INT UNSIGNED **required** AUTO_INCREMENT	Internal number of the row (made by the database)
`user_id`	INT UNSIGNED may be empty	NULL = the system itself
`action`	VARCHAR(60) **required**	e.g. payment.confirmed, prices.changed, ambassador.deleted
`target_type`	VARCHAR(30) may be empty	What kind of thing
`target_id`	INT UNSIGNED may be empty	Which one
`details`	TEXT may be empty	Details (JSON)
`ip`	VARCHAR(45) may be empty	Internet address
`created_at`	DATETIME **required**	When the row was made

LINK `user_id` must point to an existing row in `admin_users`.

Fast search on: `target_type, target_id`, `user_id, created_at`, `created_at`

`rate_limits` Rate-limit counters

WHAT IT HOLDS
Counters that stop too many forms or sign-ins from one address.

WRITTEN BY
Website forms, sign-in

READ BY
The same; cleaned nightly

Column	Type	Meaning
`bucket`	VARCHAR(120) **required**	What is counted (e.g. sign-in from one address)
`window_start`	INT UNSIGNED **required**	Unix time
`hits`	INT UNSIGNED **required** DEFAULT 0	How many times

Fast search on: `window_start`

`schema_migrations` Applied database upgrades

WHAT IT HOLDS
Which database upgrades this database already has.

WRITTEN BY
Installer (deploy)

READ BY
Installer

Column	Type	Meaning
`name`	VARCHAR(120) **required**	Name
`applied_at`	DATETIME **required**	When the upgrade was applied

5.8 The 13 ready-made lists

Numbered, read-only lists with plain column names, for anyone who opens the database (e.g. MySQL Workbench). Nothing can be changed through them. A separate **viewer** login (`tools/install.php --viewer`) sees only these lists, never the tables. The same lists are on the admin's **Lists** page.

List	What it shows	Columns
`01_registered_people`	Everyone registered for the event (paid, or a free ticket from the Owner).	Reference, Full name, Phone, Email, City, Ticket, University, Lunch day 1, Lunch day 2, Status, Paid (IQD), Paid by, Paid on, Ticket number, Arrived at the door, Accepted no-refund terms
`02_lunch_day_1`	The caterer's lists.	Full name, Phone, Ticket, Reference
`03_lunch_day_2`		Full name, Phone, Ticket, Reference
`04_students`	Registered students (the ID photos are seen in the admin).	Full name, Phone, University, Ambassador code, ID photo stored, Reference
`05_workshops`	Every workshop: price, seats, how many booked and paid, the money.	Workshop, Price (IQD), Seats, Booked, Seats left, Paid, Not paid yet, Money received (IQD), On the website
`06_workshop_people`	Who booked which workshop.	Workshop, Full name, Phone, Paid?, Amount paid (IQD), Paid how, Booked on, Reference
`07_sponsors`	Sponsorship requests.	Reference, Company, Package, Contact person, Phone, Email, Status, Price told (IQD), Agreed (IQD), Paid (IQD), Last call, Next call, Handled by, Received on
`08_exhibition`	Exhibition booth requests (separate from the sponsors).	Reference, Company, Booth type, Booth number, Contact person, Phone, Email, Status, Price told (IQD), Agreed (IQD), Paid (IQD), Last call, Next call, Handled by, Received on
`09_sponsor_calls`	Every phone call with a sponsor or exhibitor, newest first.	Company, Type, Called on, Called by, Result, Amount told (IQD), Note, Next call
`10_sponsor_packages`	The packages and booth types with their prices and places.	Type, Package, Price (IQD), Places, Confirmed, Places left, On the website
`11_payments`	Every payment attempt for event tickets.	Reference, Full name, Method, Amount (IQD), Confirmed (IQD), Status, Started, Confirmed on
`12_money_per_day`	The money per day, to compare with Psoola's report.	Day, Tickets paid, Ticket money (IQD), Held for Finance (IQD)
`13_forms_sent`	Every registration form, the moment it is sent, paid or not (newest first). Only the paid ones are registrations; the others waited for payment or expired.	Reference, Sent, Full name, Phone, Email, City, Ticket, University, Lunch day 1, Lunch day 2, Pays by, Payment, From

6. Who can see and do what

Area	Owner	Registration	Finance	Content	Check-in
Dashboard	✓	✓	✓		
Registrations (view, search)	✓	✓	✓		
Edit a registration	✓	✓			
Payments	✓		✓		
Workshops and bookings	✓	✓			
Sponsors and booths	✓	✓	✓		
Check-in at the door	✓	✓			✓
Export (CSV)	✓	✓	✓		
Student ID photos	✓	✓			
Site content (website text)	✓			✓	
Communications (emails)	✓	✓	✓		
Settings, Users, Backups, free tickets	✓				
Team alerts in Communications	✓				

Sign-in protection

- Passwords are stored only as a hash (never the password itself).
- Phone codes (two-step sign-in): **required** for Owner and Finance; other roles may switch them on, and then they are always checked. Each code works only once.
- 5 wrong passwords → account locked for 15 minutes, and the Owner is emailed. At most 20 sign-in tries per 15 minutes from one internet address.
- Signed out after 30 minutes without activity, and after 12 hours at most. Changing a password or resetting the phone signs out every old session.

7. Emails

- Every email is in **English only**. There is no setting or column for another language (removed 6 Oct 2026).
- Kinds: `ticket` (registration confirmed), `pay_now` (phone registration payment link), `sponsor_received` (to the company), `sponsor_notify` (to the team), `alert` (to the Owner).
- The text is written at sending time from the current data, so a resend after fixing an email address or name is correct.
- If sending fails it is retried after 5, 15 and 40 minutes, then marked `failed` (shown on Communications and the Dashboard).
- **Email test mode** (Settings, ON by default): every email goes to one test address instead of the real people. Turn it off at launch.
- QR code + PDF ticket in the email: switched off until the entrance check is decided (setting `ticket_qr_in_email`).
- Sending service: Resend (address ismile.krd must be verified before launch).

8. Automatic jobs (timed)

When	What it does
Every minute	Copies prices, the “close registration” switch and texts from ismile.krd; sends the emails that are due.
Every 5 minutes	Asks Psoola about payments still waiting (in case a webhook was lost); closes and deletes unpaid forms; alerts the Owner about many failed emails, a missing backup (36 h) or low disk space (< 500 MB); notices price changes.
Every night 02:30	Backs up the whole database; deletes student ID photos when their time has come; clears old rate-limit counters.

9. What is kept, and what is deleted when

Data	Kept for
Registrations, tickets, confirmed payments, workshop bookings, sponsor records	**Kept** (marked cancelled/removed, never deleted)
Audit log	**Forever** — never edited, never deleted
Unpaid website form	Can be paid for 24 hours; then closed, and deleted 48 hours later with its student ID photo
Unpaid phone form (“Pay now” link)	Can be paid for 7 days; then closed and deleted 48 hours later
Payment attempts that never reached Psoola	Deleted with the unpaid form. Attempts Psoola knows about are kept, so a late “paid” is still recorded for Finance.
Forms with money Finance must review	Kept until Finance has dealt with it
Student ID photos	Deleted for everyone 90 days after the summit (21 Nov 2026 + 90 days = **19 February 2027**); the Owner gets an email
Database backups	30 days (gzip files on the server; Owner can download them on the Backups page)
Rate-limit counters	1 day
Ambassador codes	Can be deleted by the Owner (recorded in the audit log); registrations keep the code they typed

10. Protection against abuse

Action	Limit per internet address
Registration form	60 per 10 minutes (generous: a whole class may register from one campus network)
Sponsor form	10 per 10 minutes
Start a payment	30 per 10 minutes
Payment status checks	240 per 10 minutes
Ticket QR images	120 per 10 minutes
Payment company messages (webhook)	600 per 10 minutes; every message must carry a correct signature or it is rejected and logged

Action	Limit per internet address
Admin sign-in	20 per 15 minutes

Student ID photos are checked and redrawn as clean JPEG files before saving, stored inside the database (so they are in every backup) and can be seen only in the admin after sign-in. Payment and ticket links use secret codes (`view_nonce` + the server secret), so nobody can guess another person's page.

11. Settings (Owner, Settings page)

Setting	Default	Meaning
`registration_open`	0	Registration open or closed (together with the website's “Close registration” switch)
`closed_message_en / ar / ku`	empty	Message on the website when registration is closed (website languages)
`ticket_capacity`	0	Maximum tickets; 0 = no limit set yet (the Dashboard warns)
`lunch_capacity_day1 / day2`	0	Maximum lunches per day
`email_test_mode` / `email_test_address`	1 / empty	Send every email to the test address only
`sponsor_notify_email`	empty	Who in the team is told about new sponsor requests
`summit_end` / `photo_keep_days`	2026-11-21 / 90	When student ID photos are deleted
`ticket_qr_in_email`	0	QR code + PDF ticket in the email
`pay_link_days`	7	How long a phone “Pay now” link works
`prices_seen`	(system)	Last prices seen, to notice changes — not on the Settings page

12. Test environment and live environment

Now: test (api.ismile.krd)
Online, behind a site password. Payments use a pretend gateway (no real money). The automatic test run and test clean-ups are allowed. The admin shows “Test environment”.

At launch: live
Server setting `env = live` and gateway `psoola` with Psoola's keys. The pretend gateway is refused, test tools refuse to run, the Dashboard warns if Psoola is missing. Then: delete test data, remove the site password, switch off email test mode.

13. Changing the database safely

1. `backend/database/schema.sql` is always the complete, current design; a new database is built from it alone.
2. A change to a database already in use also needs an upgrade file in `backend/database/migrations/` (named with the date). The deploy runs each file **once** and records it in `schema_migrations`. Never edit a file after it was applied.
3. Applied so far: `2026-10-06-emails-english-only.sql` (removes the email language column and the old setting).

4. Order: test site first → backup of the live database → deploy to live.

14. Backups and restore

- Nightly full export of every table and list, gzip-compressed, kept 30 days; Owner can download from Admin › Backups.
- Tested on 6 Oct 2026: restoring a backup gives back every table and list exactly, including ID photo bytes, Kurdish/Arabic text, original times and calculated fields (38 checks passed).
- The Owner is alerted if no backup was made in 36 hours.

15. Checked on 6 October 2026

Check	Result
Full scenario test on a brand-new database (registrations, payments, tickets, workshops, sponsors, emails, rules)	**215 passed, 0 failed**
Upgrade from the previous design (applied once, existing emails kept)	Passed
Admin access and pages for every role	476 passed
Backup → restore round trip	38 passed
Admin operations / phone registration fields	22 + 16 passed
Prices charged by the server	Professional 40,000 · Student 30,000 · with lunch 50,000 (test lunch price)
Emails produced by the test (16)	All English

Tests used the pretend payment gateway and saved emails to files; no real payment or real email was involved.